Práctica 3: Algoritmos Greedy

Algorítmica
DECSAI
Universidad de Granada

1. Introducción

El objetivo de esta práctica es que los estudiantes se familiaricen con los algoritmos Greedy. Para ello, se proponen dos problemas, que los estudiantes deberán resolver.

2. Problema 1: Servidores y tareas

Imagina que eres un/a arquitecto/a de redes en la nube (Cloud Arquitect). Dispones de un conjunto de servidores y una cola de tareas de procesamiento que debes asignar a esos servidores. Cada tarea debe asignarse como máximo a un servidor y a cada servidor se le puede asignar como máximo una tarea. Suponemos que tenemos el mismo número de servidores que de tareas.

Cada tarea i requiere una capacidad de cálculo mínima r_i para que el proceso se pueda ejecutar en un servidor sin agotar el tiempo de espera (time-out). Cada servidor j tiene una capacidad de procesamiento c_j. Si $c_j \geq r_i$, podemos asignar la tarea i al servidor j y la tarea se procesará correctamente. Tu objetivo es maximizar el número de tareas completadas y dar como resultado ese número máximo.

Diseña un algoritmo Greedy para resolver esta tarea. Justifica por qué con este algoritmo se obtiene la solución óptima.

Ejemplo 1. Para $r = [4, 6, 9]\,, c = [3, 5, 8]$, el número máximo de asignaciones es 2, siendo estas $(1, 2), (2, 3)$.

Ejemplo 2. Para $r = [12, 5, 8, 20, 7, 15]\,, c = [10, 6, 18, 14, 5, 22]$, el número máximo de asignaciones es 5, siendo estas $(2, 5), (5, 1), (3, 4), (1, 3), (6, 6)$.

3. Problema 2: Viajante de Comercio

Dada una lista de ciudades y las distancias entre cada par de ciudades, ¿cuál es la ruta más corta posible que pasa por cada ciudad exactamente una vez y vuelve a la ciudad de partida (tour)?

Como habéis visto en teoría, el problema del viajante de comercio (Travelling Salesman Problem, TSP) es un problema clásico en ciencias de la computación. Una solución a este problema es una permutación del conjunto de ciudades que indica el orden en que se deben recorrer. La calidad de la solución vendrá dada por la suma de las distancias entre cada par de ciudades.

Este problema pertenece a la clase de problemas NP completos, lo cual significa que, dada una

posible solución al problema, esta puede ser verificada en tiempo polinómico. Sin embargo, no conocemos algoritmos que encuentren la solución óptima en tiempo polinómico. En otras palabras, encontrar la solución óptima en cada caso requiere comprobar todas las posibles soluciones. Cuando el número de ciudades crece, esta tarea es impracticable, por lo que debemos recurrir a aproximaciones.

En esta práctica nos vamos a centrar en aproximaciones de tipo Greedy. Estos algoritmos construyen soluciones añadiendo nuevas ciudades al tour existente de acuerdo a algún criterio.

3.1. Algoritmos Greedy

Deberéis implementar y estudiar la eficiencia de dos algoritmos distintos.

Vecino más cercano. Partiendo de una ciudad inicial (elegida al azar), en cada paso se agrega como ciudad siguiente aquella que no ha sido visitada aún y que se encuentra más cercana a la actual. Este proceso se repite hasta visitar todas las ciudades.

Inserción más económica. Partimos de un sub-tour inicial que contenga tres ciudades (elegidas al azar). En cada paso, teniendo un subtour T, elegimos la ciudad a insertar como aquella que minimiza la distancia al subtour. Esta distancia se define como:

$$d(k, T) = \min_{c \in T} d(k, c).$$

Una vez que hemos elegido la nueva ciudad k, la insertamos en el subtour. Para decidir dónde insertarla, calculamos el incremento de longitud en cada lado (i, j) del subtour:

$$\Delta_{i,j} = d(i, k) + d(k, j) - d(i, j).$$

Elegimos el lado (i, j) con menor incremento de longitud y modificamos el subtour insertando k entre i y j. Este proceso se repite hasta que el subtour contiene todas las ciudades, en cuyo caso se convierte en un tour.

3.2. Material

Ficheros de prueba. Para comprobar la calidad de las soluciones obtenidas, usaremos algunos conjuntos de ciudades de prueba. Estos han sido obtenidos de la librería TSPLIB. De cada uno conocemos la solución óptima, por lo que podemos calcular su coste. Estos conjuntos se representan en ficheros `.tsp` con el siguiente formato:

```
DIMENSION: 70
EDGE_WEIGHT_TYPE : EUC_2D
NODE_COORD_SECTION
1 64 96
2 80 39
3 69 23
4 72 42
5 48 67
...
TOUR_SECTION
1
36
29
13
70
...
```

En la primera fila aparece el número de ciudades. Debajo de `NODE_COORD_SECTION` tenemos las coordenadas de cada ciudad. Debajo de `TOUR_SECTION` tenemos la solución óptima.

Código inicial. Este guión viene acompañado de un código inicial con funciones para poder leer los ficheros anteriores, calcular la matriz de distancias, y visualizar los tours obtenidos. Este código se encuentra en PRADO. A partir de este código deberéis implementar las heurísticas propuestas.

4. Tareas a realizar

Para el problema 1, debéis diseñar el algoritmo Greedy que se pide, analizar su eficiencia teórica, e implementarlo en C++.

Para el problema 2, debéis implementar las dos heurísticas propuestas, analizar su eficiencia teórica y empírica, y analizar la calidad de las soluciones que obtienen.

5. Material a entregar

Se debe entregar un fichero `.zip` que contenga:

1. El código fuente desarrollado para implementar los algoritmos.

2. Una memoria en formato `.pdf`, que incluya el nombre del equipo y los nombres de todos los miembros del equipo.

Estructura de la memoria. Se recomienda tratar cada problema en una sección diferente. Para cada problema, se recomienda seguir la siguiente estructura:

```
1. Problema 1
    1.1. Descripción
    1.2. Implementación y ejecución
    1.3. Eficiencia
    1.4. Optimalidad
2. Problema 2
    1.1. Implementación y ejecución
    1.2. Eficiencia
    1.3. Análisis de resultados
```

Los contenidos de las subsecciones del primer problema deben ser:

- **Descripción.** En esta sección, explicaréis en lenguaje natural el algoritmo Greedy obtenido. Es importante que expliquéis de forma clara y concisa cuál es la idea principal del algoritmo. Es recomendable que incluyáis una implementación en pseudocódigo.

- **Implementación y ejecución.** Debéis explicar cómo habéis implementado el algoritmo Greedy, y ofrecer ejemplos de su ejecución.

- **Eficiencia.** Debéis analizar de forma teórica la eficiencia de vuestra solución.

- **Optimalidad.** Debéis justificar si el algoritmo diseñado obtiene o no la solución óptima.

Recordad que:

- Si vuestra respuesta es afirmativa, debéis justificar de forma rigurosa por qué.
- Si vuestra respuesta es negativa, debéis dar un contraejemplo en el cual el algoritmo Greedy no obtiene la solución óptima.

Para el segundo problema:

- **Implementación y ejecución.** Debéis explicar cómo habéis implementado los algoritmos Greedy, y ofrecer ejemplos de su ejecución.
- **Eficiencia.** Debéis analizar de forma teórica y empírica la eficiencia de vuestra solución.
- **Análisis de resultados.** Debéis analizar la calidad de las soluciones que obtiene cada variante. Esto lo haréis comparando los costes de las soluciones obtenidos con los costes óptimos en cada conjunto de prueba. Este análisis debe tener en cuenta la eficiencia de cada algoritmo. Debéis discutir si merece la pena usar un algoritmo menos eficiente, a pesar de que obtenga mejores resultados. Se recomienda apoyar este análisis en gráficas y figuras. También es recomendable visualizar los tours obtenidos por cada algoritmo y compararlos visualmente con el óptimo.

La práctica deberá ser entregada a través de la plataforma PRADO, en la fecha y hora límite indicadas en la entrega. No se aceptarán prácticas entregadas con posterioridad a la fecha límite indicada.